Aston University
Computational Intelligence

Implementing the Travelling Salesman Problem

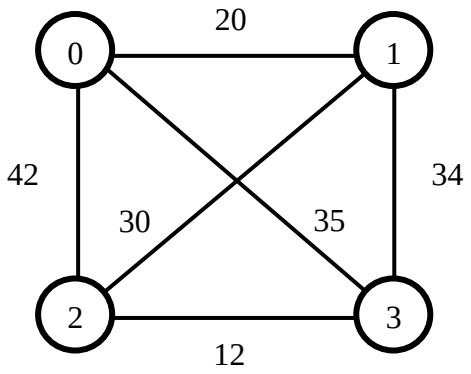
Expected Outcomes:
In this module, you will be expected to implement a variety of CI algorithms in *python*. We believe that implementing an algorithm is one of the best ways to ensure that you understand it and its details. This implementation work will also help you develop the transferrable skill of translating from an algorithm description (e.g. in pseudocode, in an academic paper) into code.

In order to test and compare those algorithms we will need to apply them to problems. In this lab, we implement the first of these problems: the Travelling Salesman Problem (TSP).

Instructions:
Recall from the lecture in Week 1 that we introduced the Travelling Salesman Problem.

Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?

Here is the example problem instance we looked at in the lecture:



Recall that there are six possible routes through this graph that satisfy the problem constraints:

0123, 0132, 0213, 0231, 0312, 0321

Since the graph is fully connected and we are anchoring the start point, this is the number of permutations of the sequence of cities minus one, i.e. $(n-1)!$.

Step 1: Data Structure for the Graph
First, implement an appropriate data structure to store the graph and its weights. I suggest something simple. (i.e. don't create an object for every node). A common way to do this is to use an **adjacency matrix**: a 2D matrix in which entry i, j gives the length from city i to city j. For example, for the problem above, we might have the following:

```
adjacency_matrix[0][1] = 20 # The distance from city 0 to 1 is 20.
```

Populate this data structure with the example graph given above.

Question: how will your data structure grow according to the number of cities? Make sure you can instantiate the data structure for problems of different sizes (i.e. different numbers of cities).

Step 2: Defining and Evaluating Routes
Next, we need to be able to define a route (e.g. 0123) in our code and evaluate it – calculating its length. Before diving into coding this, think about the algorithm that that function will need to follow.

After completing this step, we should be able to write statements something like the following:

```
adjacency_matrix = # adjacency matrix defined here
route = # route defined here
cost = evaluate_tsp(adjacency_matrix,route)
```

and have the `cost` variable store the true cost of the route. Remember to take into account the fact that a tour needs to return to the starting city. You could do so either in your definition of a route or in your definition of the `evaluate_tsp` function. Just remember to be consistent!

Double-check your implementation by calculating the cost of a route by hand and making sure that your implementation produces that same cost.

Step 3: Generating Random Routes
In future labs, it will be useful for you to be able to generate random (legal!) routes. Implement a function to do this. Once defined, you should be able to write something like the following:

```
n_cities = 4 # The TSP with 4 cities
route = random_route(n_cities)
```

and have a random, legal, route generated. You could take advantage of the fact that if you start out with any route and then shuffle the elements into random positions, the resulting route will also be legal. The python method `random.shuffle` might come in handy.

Make sure that your `random_route` function works for numbers of cities other than 4. You will be solving bigger problems than this!

Generate a few random routes and evaluate them and see the range of costs that exist.

Step 4: Try a Bigger Problem Instance

Download the file `ulysses16.csv` from Blackboard and take a look inside. It contains two columns: for each row, these represent the city's x and y coordinates on a map. Assume that the graph is fully connected (i.e. you can reach each city from every other) and that the cost associated with travelling between two cities is Euclidean distance between them.
i.e. to work out the cost of travelling between city i and city j, you need to use Pythagoras's theorem:

$$distance_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

where $distance_{i,j}$ is the distance from city i to city j, x_i, y_i, x_j and y_j are the x and y coordinates of cities i and j.

It should be reasonably straightforward to write a short program to read in this file (perhaps using the `pandas.read_csv` function in python) and use the above calculation to populate your data structure from Step 1.